

DOCUMENTO DE ARQUITETURA

CheckFlow

Plataforma de checklists com verificação em tempo real, arquitetura unificada e capacidade offline. Pensada para evoluir em direção a IoT e IA sem reescrita.

Versão: 1.0 · rascunho de arquitetura

Data: junho de 2026

Repositório: github.com/ctmf81/checkflow

Escopo deste documento: visão de arquitetura, camadas, fluxo de dados e stack tecnológico

1. Sumário executivo

O CheckFlow é um sistema de checklists configuráveis com execução e verificação em tempo real. A arquitetura abandona a separação tradicional entre ferramenta de front-end e back-end no-code (modelo Bubble + Xano) em favor de uma estrutura unificada, versionada integralmente em Git, onde as regras de negócio são código aberto, testável e auditável.

A decisão central de design é tratar cada resposta de checklist como um **evento imutável**. Essa escolha, tomada na fundação do banco de dados, é o que permite que o mesmo sistema atenda três cenários sem mudança estrutural: execução manual hoje, ingestão automática de sensores IoT amanhã, e validação por IA em seguida. Para o sistema, uma leitura de sensor é apenas mais um evento — idêntico em natureza a um operador marcando um item.

Princípios norteadores

- **Tudo no Git.** Regras de negócio, schema do banco e permissões vivem em código versionado, não em cliques numa interface.
- **Uma fonte da verdade.** Toda lógica reside na API; os clientes (web, e futuramente mobile) são apenas telas que a consomem.
- **Web primeiro.** O MVP é um único aplicativo web (PWA), capaz de configurar, executar e relatar — com validação funcionando offline.
- **Fundação pronta para o futuro.** A porta de ingestão para IoT e IA existe desde a primeira versão, mesmo que ativada depois.

Em uma frase: uma arquitetura enxuta de construir agora (apenas duas camadas próprias: o PWA e a API), apoiada em serviço gerenciado para dados e tempo real, e desenhada para que IoT e IA sejam adições — não reescritas.

2. Visão geral da arquitetura

O sistema se organiza em quatro camadas. Apenas as duas centrais (clientes e API) são construídas pela equipe; a camada de dados é serviço gerenciado e a camada de fontes automáticas é roadmap.

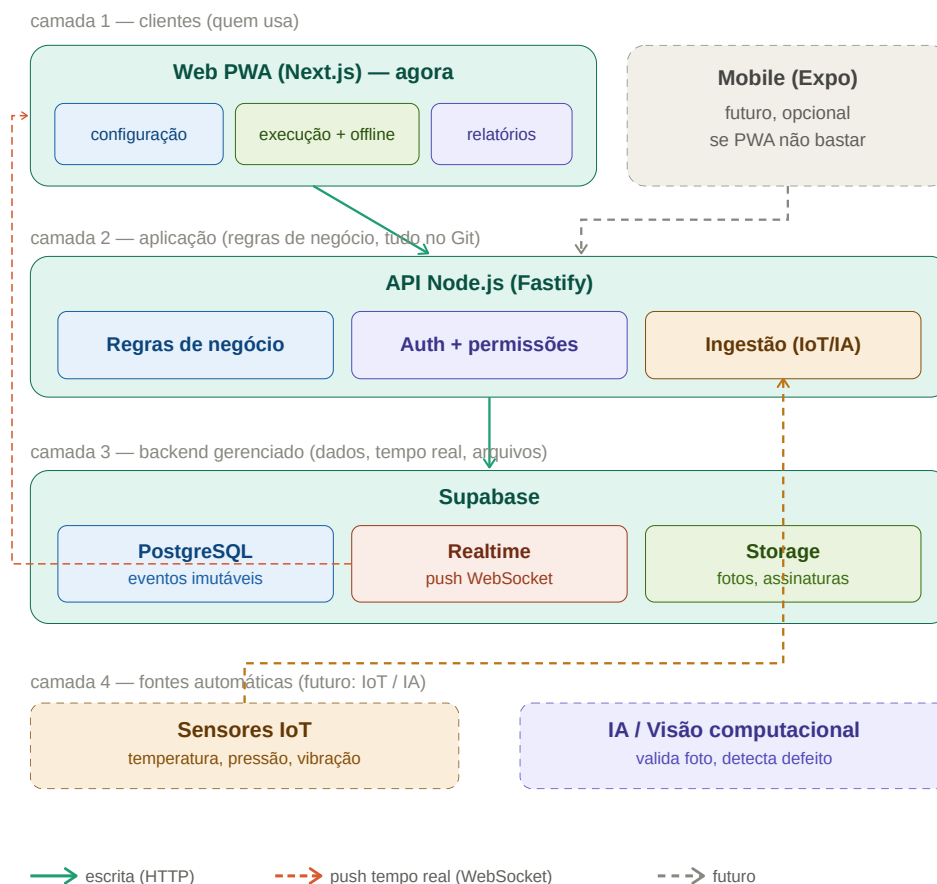


Figura 1 — As quatro camadas do CheckFlow e os fluxos de dados entre elas.

3. As camadas em detalhe

1 Clientes

web agora · mobile futuro

O ponto de execução principal é o **web (Next.js como PWA)** — um aplicativo instalável, com cache offline, que cobre os três usos: configurar checklists, executá-los e visualizar relatórios. A validação de regras acontece localmente no dispositivo, dispensando internet para a maioria das verificações (campos obrigatórios, intervalos, formatos, lógica condicional).

O **mobile (Expo)** é adiado e opcional. Como toda regra vive na API, ele será apenas mais um cliente da mesma fonte da verdade quando — e se — for necessário para cenários de campo sem conexão que o PWA não cubra.

2 Aplicação (API)

construída · no Git

O cérebro do sistema, e a camada que substitui o Xano. Uma **API Node.js com Fastify** concentra as regras de negócio, a autenticação com controle granular de permissões, e a porta de ingestão por onde sensores e IA escreverão dados no futuro. Todo o código é TypeScript versionado em Git — testável, revisável por pull request e auditável.

3 Backend gerenciado

serviço · não construído

O **Supabase** entrega pronto o que não precisa ser construído: o **PostgreSQL** armazena as respostas como eventos imutáveis; o **Realtime** empurra mudanças via WebSocket a todos os dispositivos conectados, sem polling; e a autenticação (**Auth**) já vem integrada. O armazenamento de arquivos começa no Supabase Storage pela simplicidade e migra para o **Cloudflare R2** conforme o volume de fotos cresce (ver seção 5.1).

4 Fontes automáticas

roadmap

A camada que define o posicionamento de longo prazo. **Sensores IoT** alimentam itens de checklist automaticamente (uma leitura de temperatura responde sozinha a um item), e modelos de **IA / visão computacional** validam fotos e detectam defeitos. Ambos escrevem na mesma porta de ingestão da API — para o sistema, são apenas mais uma origem de eventos.

3.1. Fluxo de uma resposta em tempo real

O ciclo completo, do toque do operador à atualização nas telas de todos os demais usuários, ocorre em menos de um segundo:

#	Etapa	O que acontece
1	Operador marca	Item é preenchido no app; a validação local roda na hora.
2	API valida	A regra de negócio e a permissão do usuário são verificadas no servidor.
3	Grava evento	A resposta é registrada no PostgreSQL como evento imutável (não sobrescreve nada).
4	Realtime dispara	Um trigger no banco notifica o serviço de tempo real sobre a mudança.
5	Todos veem	A alteração é empurrada via WebSocket; as telas atualizam sem recarregar.

Por que isso importa: a diferença entre *push* e *polling* é a do garçom que volta para avisar que o prato ficou pronto, versus você levantando da mesa a cada minuto para perguntar na cozinha. O segundo desperdiça recursos e sempre chega atrasado. Esse mesmo fluxo serve os três cenários: hoje o operador dispara a etapa 1; amanhã um sensor a dispara; depois a IA valida na etapa 2.

4. Stack tecnológico

Cada escolha responde a uma necessidade específica do sistema. O conjunto privilegia unificação, tempo real nativo e mínimo de infraestrutura para gerenciar.

Camada	Tecnologia	Por que
Cliente web	Next.js (PWA)	Front e parte do back unificados; instalável e com cache offline para validação local.
Cliente mobile	Expo (React Native)	Futuro. Reaproveita conhecimento de React; consome a mesma API.
API	Node.js + Fastify	Leve e rápido; regras de negócio como código testável no Git, independente do front.
Linguagem	TypeScript	Tipos compartilhados entre web, mobile e API; menos erros em tempo de execução.
Banco de dados	PostgreSQL (Supabase)	Relacional robusto; suporta o modelo de eventos imutáveis e consultas analíticas.
Tempo real	Supabase Realtime	WebSocket nativo sem infraestrutura extra; propaga mudanças automaticamente.
Arquivos	Cloudflare R2	Storage compatível com S3 sem taxa de egresso — decisivo para um sistema com muitas fotos visualizadas. Inicia no Supabase Storage e migra para R2 quando o volume justificar.
Autenticação	Supabase Auth	JWT e login social (Google, Microsoft) prontos; integra com as permissões da API.

4.1. Sobre custo e escala

O plano gratuito do Supabase suporta 200 conexões simultâneas em tempo real e 2 milhões de mensagens por mês — suficiente para o MVP e validação inicial. O plano Pro (US\$ 25/mês) eleva para 500 conexões simultâneas e remove a pausa por inatividade, sendo o patamar recomendado para produção. Acima de dezenas de milhares de usuários conectados ao mesmo tempo, o canal de tempo real pode migrar para serviços dedicados (Ably, Pusher) sem alterar o restante da arquitetura.

5. Hospedagem e custos

A filosofia é de austeridade financeira com crescimento proporcional: começar barato e só assumir custos maiores quando houver receita que os justifique. A infraestrutura se concentra em **dois fornecedores** — Supabase e Railway — mais o Cloudflare R2 para arquivos. Nunca se paga por capacidade ainda não utilizada.

5.1. Por que dois fornecedores, e não cinco

Quase todas as peças do sistema poderiam rodar em qualquer provedor. A concentração no Supabase existe por um motivo específico: ele entrega **tempo real e banco gerenciado prontos**, sem necessidade de construção. Como a verificação em tempo real é o requisito central do CheckFlow, ter isso resolvido economiza semanas de desenvolvimento. O Railway hospeda a API (mantendo processos sempre ligados, ao contrário de plataformas serverless que não servem bem WebSocket).

O **storage é a única peça deliberadamente externa**. O Cloudflare R2 é compatível com a API do S3, porém sem a taxa de egresso — a AWS cobra US\$ 0,09 por GB transferido do S3 para a internet, enquanto a R2 não cobra egresso. Para um sistema de checklist onde fotos de inspeção são visualizadas repetidamente em relatórios, o custo de *exibir* arquivos pode superar o de armazená-los; a R2 elimina esse risco. A recomendação prática é iniciar no Supabase Storage pela simplicidade e migrar para a R2 quando o volume crescer — como ambos falam a API do S3, a migração troca apenas configuração, não código.

5.2. Custo por fase de crescimento

Fase	Quando	Custo/mês	Composição
1 • Validação	MVP até ~10 empresas	~US\$ 10	Railway Hobby (US\$ 5) + Supabase Free + frontend gratuito (Cloudflare Pages). Atenção: o plano Hobby da Vercel proíbe uso comercial — ao cobrar clientes, usar Cloudflare Pages ou Vercel Pro.
2 • Tração	~10 a 100 empresas	~US\$ 60	Supabase Pro (US\$ 25, remove pausa por inatividade e adiciona backups) + Railway/Render (US\$ 20–25) + frontend. O salto para o Supabase Pro é o gasto prioritário ao entrar em produção.
3 • Escala	centenas de empresas	US\$ 300+	Compute maior, possível autoescala da API, e tempo real dedicado (Ably/Pusher) se necessário. Aqui o custo é fração da receita — a austeridade dá lugar à proporcionalidade.

Princípio de escala sem reescrita: todas as fases usam as mesmas tecnologias — o mesmo Next.js, a mesma API Node, o mesmo Supabase. Sobe-se de plano, nunca se troca de stack. O

crescimento é uma questão de orçamento, não de reengenharia.

6. Estratégia offline (PWA)

A validação se divide em dois tipos. O PWA resolve o primeiro integralmente sem internet; o segundo é tratado por sincronização posterior.

Funciona offline (validação local)	Exige internet (validação contra o servidor)
Campo obrigatório não preenchido	Verificar se um código de ativo existe no sistema
Número fora do intervalo permitido	Conferir unicidade global (já foi usado hoje?)
Formato inválido (e-mail, data, CPF)	Permissão em tempo real (quem pode aprovar agora)
Lógica condicional entre itens	Leitura de sensor IoT ao vivo
Evidência (foto/assinatura) anexada	Validação por IA da foto enviada

O padrão de operação: o app valida localmente e registra a resposta na hora; as verificações que dependem do servidor ficam marcadas como pendentes e são resolvidas quando a conexão retorna. Em checklists de inspeção isso funciona bem, pois o operador registra o que *observa* — e a observação não muda conforme haja ou não conexão. A temperatura é a temperatura; a foto é a foto.

Conclusão de design: como as validações críticas do CheckFlow são majoritariamente locais, o PWA atende ao requisito de uso offline sem a necessidade imediata de um aplicativo nativo. O mobile permanece como evolução possível, não como pré-requisito.

7. Decisões-chave que sustentam o futuro

Três decisões precisam estar corretas desde a primeira versão do banco, pois são caras ou impossíveis de adicionar depois — a analogia é a fundação e o encanamento de um prédio: dá para mudar a pintura e os andares, não a base.

- **Eventos imutáveis (event sourcing).** Cada resposta é um registro novo com autor, valor e timestamp, em vez de sobrescrever um valor único. É o que viabiliza análise preditiva, auditoria real e ingestão contínua de sensores.
- **Origem da resposta tipada.** Cada evento carrega quem respondeu — *humano*, *sensor* ou *IA*. O mesmo checklist serve inspeção manual hoje e monitoramento automático amanhã, sem mudar o modelo.
- **Multi-tenant e ingestão desde o dia um.** O isolamento entre empresas e a porta de entrada de dados externos existem desde a primeira migration, evitando uma reescrita do banco no futuro.

8. Recomendação final

Síntese das decisões. O princípio que costura tudo: **escopo mínimo agora, fundação certa para o futuro** — construir pouco, sobre uma base que aguenta crescer sem demolição.

8.1. O que construir

- **Frontend:** Next.js como PWA, com foco no web. Cobre configuração, execução (com validação offline) e relatórios num único app.
- **API:** Node.js + Fastify em TypeScript, concentrando todas as regras de negócio, versionadas em Git num monorepo.
- **Mobile:** adiado. Só entra se algum cenário de campo exigir além do que o PWA oferece — e, quando entrar, será apenas mais um cliente da mesma API.

8.2. Onde hospedar

Componente	Serviço
Banco de dados, tempo real e autenticação	Supabase
API (regras de negócio)	Railway
Frontend PWA	Cloudflare Pages (ou Vercel Pro)
Storage de fotos e assinaturas	Supabase Storage → Cloudflare R2

São essencialmente dois fornecedores de núcleo (Supabase e Railway), mais o R2 para arquivos. A escolha do R2 para storage é definitiva: storage compatível com S3, sem taxa de egresso, ideal para o alto volume de visualização de fotos típico de um sistema de inspeção.

8.3. Como crescer

Avançar de fase apenas quando a fase atual apertar — e isso só ocorre quando já há clientes pagantes que justificam o gasto. Inicia em ~US\$ 10/mês (validação), passa a ~US\$ 60/mês ao virar produto pago (com o Supabase Pro como prioridade), e chega a centenas de dólares apenas na escala, quando o custo já é fração da receita. Em todas as fases, a mesma stack — sobe-se de plano, nunca se reescreve.

Próximo passo: descer ao modelo de dados — as tabelas do PostgreSQL que materializam o event sourcing, a origem tipada das respostas e o isolamento multi-tenant descritos na seção 7. É onde as regras de negócio começam a ser definidas concretamente.